

EvidenceGraph -- Problem Statement

How I arrived at the problem I'm solving, and why the more obvious versions of it didn't survive contact with prior art. Written for the Razorpay Track 02 evaluators.

Before I give you the final problem statement, I want to walk you through how I got there. I didn't start here -- I started with a bigger, more obvious idea, and every version of it got weaker the harder I checked it against what already exists in fraud research and in the patent record. What's left is narrower than where I started. I'd rather hand you a small problem I can defend end-to-end than a large one that falls over on your first follow-up question, so this document is the actual sequence of ideas I rejected, in order, and why.

A note on method, since it matters for how much weight to put on the "why we rejected it" sections below: every rejection here is backed by something specific and checkable -- a granted patent or filing (checked on Google Patents / USPTO), a named 2025-2026 security or fraud-research paper, or a real vendor's own product documentation. Where I name a patent number or a paper below, that citation is the actual reason the previous idea got crossed out, not decoration added afterwards.

1. Where I started

My first instinct was the obvious one: payment fraud evolves faster than the systems built to catch it. Most risk engines score a transaction against patterns learned from history -- past fraud, transaction amount, device, IP, location, merchant behaviour, velocity, customer history, behavioural patterns. Fraudsters don't hold still, and a system trained on last year's fraud is structurally behind this year's. So my first framing was: how do you proactively surface a new fraud strategy before it turns into a loss, instead of after?

It's a good sentence to say out loud. It also turned out to already be someone else's product. Mastercard holds a patent covering synthetic fraudulent transactions: the system generates synthetic transactions, enriches them with velocity information, runs them through a simulated authorisation model, and evaluates the vulnerabilities that surface. On top of that there is active academic research into adversarial fraud and fraud-model evasion, and commercially, vendors are already selling AI-driven fraud red-teaming products built around exactly this idea. I wasn't going to out-execute that inside a hackathon, and more to the point, the gap wasn't actually open -- so I dropped it.

2. Trying to go one level further: optimising the response

Once a system can find a new attack, the next question writes itself: can it also work out the best way to close it? That became something close to adaptive risk-policy optimisation -- when a new vulnerability shows up, how does a risk team change its policy without either missing more fraud or creating friction for a lot of legitimate customers? An AI that picks the safest policy change felt genuinely useful.

Same outcome, though. BetterBehave already describes exactly this kind of policy experimentation -- running a current policy against simulated variants and tracking the trade-off across fraud reduction, friction, and false positives, with policy variants validated through live experimentation rather than simulation alone. Darwinium's products already pair adversarial fraud testing with remediation and optimisation in the same pipeline. "Find the best policy change automatically" wasn't differentiated enough either, so I moved on again.

3. Going deeper: what if the signals themselves can't be trusted?

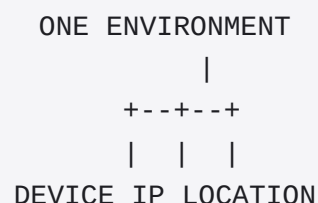
Both of my first two ideas assumed the underlying signals -- device, IP, location, velocity, merchant, customer history, payment method, amount, behaviour -- stay meaningful over time. They don't. User behaviour shifts, devices change, attacker tactics adapt, and a signal that used to predict fraud well can quietly stop meaning what it used to mean. So the next framing was about trust in signals over time: can a risk system continuously tell which of its signals are still predictive, and adjust how much weight it puts on each one?

This one felt like real research territory, and it partly is -- but it isn't unclaimed. Mastercard holds a granted patent, US 11,410,187 B2, that explicitly covers detecting feature drift in fraud-prevention models and generating a hardened replacement model in response. Current fraud research is also heavily invested in dynamic graphs, concept drift, and time-aware detection more broadly -- this isn't a niche corner of the literature, it's a major active thread in it. Rejected again, for the same underlying reason as the first two: solid engineering, but not an open gap.

4. The question that actually held up

Here's where the thinking changed shape. Instead of asking whether any single signal is trustworthy, I started asking a different question: when several signals agree, do they actually agree independently, or are they just saying the same thing three different ways?

Picture a payment with a trusted device, a trusted IP, a normal location, a normal merchant, and a normal customer history. A risk engine reading that sees five signals pointing the same direction and treats it as five independent votes for "safe." But device, IP, and location can all trace back to one environment -- the same network, the same physical setup, the same session:



If they do, the system doesn't actually have five pieces of evidence. It has one piece of evidence, counted five times, wearing different labels.

That's the specific failure mode I decided was worth building around: correlated or redundant signals manufacturing a confidence the underlying evidence doesn't actually support.

5. Stress-testing evidence independence too

The honest answer is that "evidence graphs" and "correlated evidence" are not new concepts either. NIST has used evidence graphs to model dependencies between pieces of security evidence and reason about the relationships between them for well over a decade. There's a 2026 paper on financial-statement fraud that explicitly constructs evidence graphs with supporting and contradicting evidence, and current transaction-fraud research already uses heterogeneous graphs linking accounts, devices, addresses, and other entities to each other.

So "build an evidence graph for fraud" is not, on its own, a pitch. But this rejection was different in kind from the first three. I didn't need to throw the idea away -- I needed to go one level more abstract, because the graph itself was never the interesting part. The interesting part was the question the graph was being asked to answer.

6. The question that survived

Here's what I landed on. The question worth asking isn't "is this transaction fraudulent," "is this signal risky," "has the model drifted," or "can an attacker get around the model." Those are all questions about the transaction or about the model. The question I think is genuinely underserved is about the decision itself: how trustworthy is the evidence that a risk system just used to reach the conclusion it reached?

That's a different axis entirely. It isn't asking the risk engine to be more accurate. It's asking whether the confidence a risk engine reports about its own decision is earned by genuinely independent, consistent evidence -- or borrowed from evidence that only looks like several things when it's really one thing.

7. The problem, stated properly

Modern payment-risk systems make real-time authorisation decisions by combining many signals -- device, network, location, transaction behaviour, merchant history, customer history. But more signals agreeing doesn't automatically mean more independent evidence. Signals can be correlated, redundant, contradictory, stale, or traceable back to the same underlying entity or event, and a risk engine that treats all of them as equally independent can end up very confident about a decision for reasons that don't actually hold up.

It gets harder because you often can't know at decision time whether the evidence was actually good. Whether a payment turns out to be disputed, charged back, or confirmed fraudulent shows up later -- sometimes much later -- so the system frequently can't tell, in the moment, whether what it relied on was reliable. This delayed-and-incomplete-label problem is well

documented in fraud-modelling research; it's not a solvable-in-hindsight footnote, it's structural.

Put together: there's a gap between producing a risk decision and knowing whether the evidence behind it deserves the confidence it's given. Existing systems are built to answer "what is the risk." Almost none of them are built to answer "how trustworthy is the evidence behind that risk number" -- and that second question is the one I decided to spend this build on.

8. If you remember one sentence

Payment systems can tell you how risky a transaction looks. They don't necessarily tell you how trustworthy the evidence behind that risk assessment actually is.

That's the whole pitch.

9. Why I think this matters to Razorpay specifically

I want to be precise about what I'm not saying. I'm not saying Razorpay's fraud detection is inadequate -- Shield already runs large numbers of signals through AI/ML-based risk analysis, and it's good at what it's built to do. What I am saying is narrower: as a risk system gets more sophisticated and folds in more signals, the question of whether those signals are independently informative gets more important, not less, and I don't think it's a question most risk engines are built to ask about themselves. This sits next to Shield, as a check on the evidence feeding it -- it isn't a replacement for it.

10. Where the gap actually sits

The pipeline every risk system runs looks roughly like this:

```
SIGNALS -> RISK MODEL -> RISK SCORE -> DECISION
```

What's missing is a step between the signals and the model -- a check on whether the signals feeding the model actually constitute independent evidence, before that evidence gets compressed into a single number:

```
SIGNALS -> [ EVIDENCE INTEGRITY ] -> RISK MODEL -> RISK SCORE -> DECISION
```

That missing step is what I'm calling evidence integrity, and it's what EvidenceGraph checks.

11. What I mean by "evidence integrity"

I want to be specific here, because on its own "evidence integrity" could mean almost anything.

I mean it as six distinct failure modes, and a system that claims to check integrity needs to check all six, not just the one that's easiest to compute:

Dependency -- two signals that look separate but trace back to the same underlying source.

Redundancy -- several signals that each add almost no new information once you already have the others.

Contradiction -- one signal saying safe while another, equally authoritative one, says suspicious.

Staleness -- evidence that used to be predictive but no longer reflects current behaviour.

Concentration -- a decision that looks well-supported but actually rests on one evidence cluster wearing several hats.

Outcome uncertainty -- not yet knowing whether the decision this evidence supported was actually correct.

Evidence integrity, then, is the degree to which the evidence behind a risk decision is independent, internally consistent, still temporally relevant, and eventually accountable to what actually happened.

12. The problem statement I'm building against

Putting all of this together, here is the version I actually built EvidenceGraph to answer:

Payment-risk systems increasingly rely on large numbers of real-time signals to make automated decisions, but a high count of agreeing signals can create an illusion of certainty when those signals are correlated, redundant, contradictory, stale, or derived from the same underlying source. At decision time, a risk system can know how risky a payment looks without knowing how trustworthy the evidence behind that assessment actually is.

This is made worse by delayed ground truth -- whether a payment turns out to be fraudulent, disputed, or legitimate is often only known well after the original decision, so risk teams don't have a continuous way to check not just whether their decisions were right, but whether the evidence behind them was genuinely independent and predictive at the time it was used.

The result is a gap between a risk score and the trustworthiness of the evidence supporting it. Closing that gap matters because a sophisticated risk system doesn't fail by being unsophisticated -- it fails by being confidently wrong, when several correlated or compromised signals happen to reinforce the same incorrect conclusion.

13. Why I'm showing you the rejected ideas, too

I could have handed you section 12 on its own and skipped everything before it. I chose not to, because I think the four dead ends matter almost as much as the destination. Each one sounded strong the moment I wrote it down, and each one had already been built by someone with a lot

more resources than a hackathon team. The only way I found a gap that was actually still open was by checking my own best idea against prior art every single time it looked good, rather than stopping at the first version that sounded impressive. That process -- patents checked, papers read, vendor products actually looked at -- is, I think, as much a part of the argument for this problem statement as the statement itself.